Rachunek macierzowy - laboratorium 1

Jakub Karczewski

20.02.2026

1 Treść zadania

1. Wybrać ulubiony język programowania. Osobiście wybrałem język Python
2. Zaimplementować rekurencyjne mnożenie macierzy kwadratowych o rozmiarze $2^k \times 2^k$:
   - metodą Binéta,
   - metodą Strassena.
3. Zaimplementować wariant mieszany:
   - dla macierzy o rozmiarze mniejszym lub równym $2^l \times 2^l$ stosować algorytm Strassena,
   - dla macierzy o rozmiarze większym niż $2^l \times 2^l$ stosować algorytm Binéta.
4. Podczas działania algorytmów zliczać liczbę operacji zmiennoprzecinkowych (dodawanie, odejmowanie, mnożenie, dzielenie) wykonywanych w trakcie mnożenia macierzy i potem przedstawić na wykresie dla różnych wartości k i l.

2 Pseudokody algorytmów

2.1 Algorytm Strassena

Algorithm 1 Algorytm Strassena z liczeniem FLOPS

function $\textsc{Strassen}(A, B)$
  $adds \leftarrow 0$
  $muls \leftarrow 0$
  return $\textsc{StrassenRec}(A, B, adds, muls)$
end function

function $\textsc{StrassenRec}(A, B, adds, muls)$
  if $|A| = 1$ **then**
    $muls \leftarrow muls + 1$
    return $(A \cdot B,\ adds,\ muls)$
  end if
  Podział macierzy:
  $A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$
  $m \leftarrow \frac{n}{2}$

  Obliczenie macierzy pomocniczych:
  $M_1 \leftarrow \textsc{StrassenRec}(A_{11} + A_{22},\ B_{11} + B_{22})$
  $adds \leftarrow adds + 2m^2$
  $M_2 \leftarrow \textsc{StrassenRec}(A_{21} + A_{22},\ B_{11})$
  $adds \leftarrow adds + m^2$
  $M_3 \leftarrow \textsc{StrassenRec}(A_{11},\ B_{12} - B_{22})$
  $adds \leftarrow adds + m^2$
  $M_4 \leftarrow \textsc{StrassenRec}(A_{22},\ B_{21} - B_{11})$
  $adds \leftarrow adds + m^2$
  $M_5 \leftarrow \textsc{StrassenRec}(A_{11} + A_{12},\ B_{22})$
  $adds \leftarrow adds + m^2$
  $M_6 \leftarrow \textsc{StrassenRec}(A_{21} - A_{11},\ B_{11} + B_{12})$
  $adds \leftarrow adds + 2m^2$
  $M_7 \leftarrow \textsc{StrassenRec}(A_{12} - A_{22},\ B_{21} + B_{22})$
  $adds \leftarrow adds + 2m^2$

  Składanie wyniku:
  $C_{11} \leftarrow M_1 + M_4 + (M_7 - M_5)$
  $adds \leftarrow adds + 3m^2$
  $C_{12} \leftarrow M_3 + M_5$
  $adds \leftarrow adds + m^2$
  $C_{21} \leftarrow M_2 + M_4$
  $adds \leftarrow adds + m^2$
  $C_{22} \leftarrow (M_1 - M_2) + (M_3 + M_6)$
  $adds \leftarrow adds + 3m^2$

  return $(C,\ adds,\ muls)$ 2
end function

2.2 Algorytm Bineta

Algorithm 2 Algorytm Binéta z przełączaniem na Strassena

function $\textsc{Binet}(A, B, l)$
  $adds \leftarrow 0$
  $muls \leftarrow 0$
  return $\textsc{BinetRec}(A, B, l, adds, muls)$
end function

function $\textsc{BinetRec}(A, B, l, adds, muls)$
  if $|A| \leq 2^l$ **then**
    return $\textsc{Strassen}(A, B, adds, muls)$
  end if
  if $|A| = 1$ **then**
    $muls \leftarrow muls + 1$
    return $(A \cdot B,\ adds,\ muls)$
  end if
  Podział macierzy:
  $$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$
  $m \leftarrow \frac{n}{2}$

  Rekurencja:
  $(X_1, adds, muls) \leftarrow \textsc{BinetRec}(A_{11}, B_{11}, l, adds, muls)$
  $(X_2, adds, muls) \leftarrow \textsc{BinetRec}(A_{12}, B_{21}, l, adds, muls)$
  $C_{11} \leftarrow X_1 + X_2$
  $adds \leftarrow adds + m^2$

  $(X_3, adds, muls) \leftarrow \textsc{BinetRec}(A_{11}, B_{12}, l, adds, muls)$
  $(X_4, adds, muls) \leftarrow \textsc{BinetRec}(A_{12}, B_{22}, l, adds, muls)$
  $C_{12} \leftarrow X_3 + X_4$
  $adds \leftarrow adds + m^2$

  $(X_5, adds, muls) \leftarrow \textsc{BinetRec}(A_{21}, B_{11}, l, adds, muls)$
  $(X_6, adds, muls) \leftarrow \textsc{BinetRec}(A_{22}, B_{21}, l, adds, muls)$
  $C_{21} \leftarrow X_5 + X_6$
  $adds \leftarrow adds + m^2$

  $(X_7, adds, muls) \leftarrow \textsc{BinetRec}(A_{21}, B_{12}, l, adds, muls)$
  $(X_8, adds, muls) \leftarrow \textsc{BinetRec}(A_{22}, B_{22}, l, adds, muls)$
  $C_{22} \leftarrow X_7 + X_8$
  $adds \leftarrow adds + m^2$

  return $(C,\ adds,\ muls)$
end function

3 Fragmenty kodu

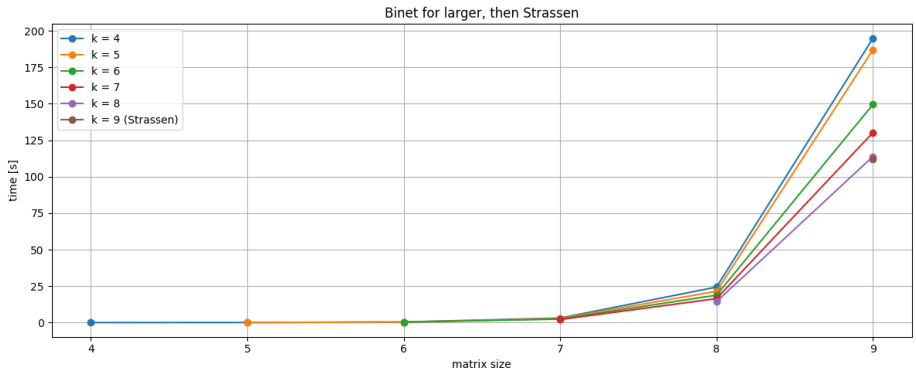
3.1 Kod do wykresu liczby operacji zmiennoprzecinkowych

```
min_k, max_k = 4, 8
flops_binet = np.zeros((max_k + 1, max_k + 1))
random_matrix = np.random.randint(0, 2, (2**max_k, 2**max_k))
for k in range(min_k, max_k+1):
    for l in range(2, k+1):
        print(l, end = " ")
        time_start = time.time()
        size = 2**k
        A = random_matrix[:size, :size]
        m2, adds_mixed, muls_mixed = binet_alg(A, A, k, l)
        flops_binet[k,l] = adds_mixed + muls_mixed

df2 = pd.DataFrame(flops_binet)
fig, axes = plt.subplots(1, 1, figsize=(12, 5), sharey=True)

for k in range(min_k, max_k+1):
    axes.plot(df2.loc[k:, k], marker='o', label=f"k = {k}{" (
        Strassen)" if k == max_k else ""}")

axes.set_xlabel("matrix size")
axes.set_ylabel("number of Flops in 10^6 flops")
axes.set_xticks(range(min_k, max_k + 1))
axes.set_title("Flops for Binet, with Strassen threshold")
axes.legend()
axes.grid(True)

plt.tight_layout()
plt.show()
```

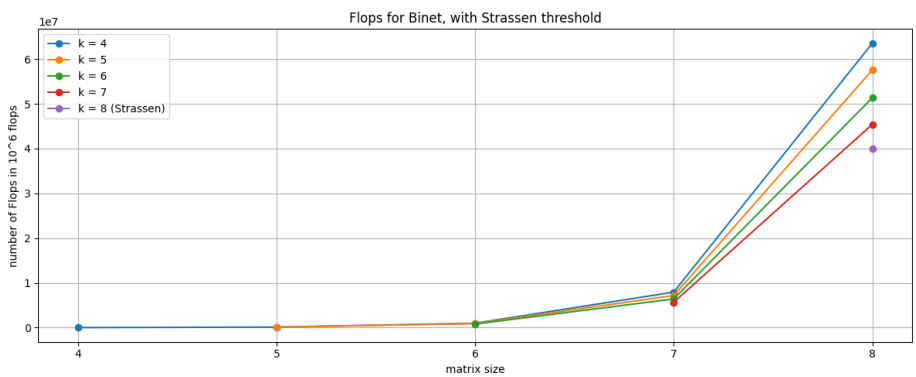
Kod dla wykresu czasów jest analogiczny

4 Wykresy

4.1 Czasy



4.2 Liczba operacji zmiennoprzecinkowych



5 Złożoność obliczeniowa

Złożoność obliczeniowa analizowanych algorytmów można określić na podstawie liczby wykonywanych operacji arytmetycznych w zależności od rozmiaru macierzy.

5.1 Algorytm Binéta

Dla klasycznego algorytmu Binéta, który wykorzystuje rekurencyjny podział macierzy, złożoność obliczeniowa wynosi:

$$T(n) = 8 \cdot T\left(\frac{n}{2}\right) + \mathcal{O}(n^2)$$

co prowadzi do złożoności asymptotycznej:

$$T(n) = \mathcal{O}(n^3)$$

Oznacza to, że liczba operacji rośnie sześciennie wraz ze wzrostem rozmiaru macierzy, co znajduje odzwierciedlenie w wynikach eksperymentalnych.

5.2 Algorytm Strassena

Algorytm Strassena redukuje liczbę mnożeń macierzy kosztem zwiększenia liczby dodawań. Jego złożoność obliczeniowa opisana jest zależnością:

$$T(n) = 7 \cdot T\left(\frac{n}{2}\right) + \mathcal{O}(n^2)$$

W konsekwencji złożoność asymptotyczna wynosi:

$$T(n) = \mathcal{O}(n^{\log_2 7}) \approx \mathcal{O}(n^{2.81})$$

Oznacza to, że algorytm Strassena jest asymptotycznie bardziej wydajny od klasycznego algorytmu Binéta dla dużych rozmiarów macierzy.

5.3 Wariant mieszany

W przypadku zastosowania podejścia mieszanego, w którym dla mniejszych podproblemów stosowany jest algorytm Strassena, a dla większych algorytm Binéta (lub odwrotnie), rzeczywista złożoność obliczeniowa zależy od przyjętej wartości progu przełączania. Odpowiedni dobór tego parametru pozwala zminimalizować całkowitą liczbę operacji oraz poprawić praktyczną wydajność algorytmu.

6 Wnioski

Na podstawie przeprowadzonych eksperymentów można sformułować następujące wnioski:

1. W analizowanych eksperymentach rozmiar macierzy był zależny od parametru k, przy czym macierze miały wymiary $2^k \times 2^k$. Oznacza to, że dla na przykład dla $k = 2$ wykonywane było mnożenie macierzy o wymiarach 4×4. Wraz ze wzrostem wartości k obserwowany jest wykładniczy wzrost rozmiaru danych wejściowych.

2. Liczba wykonanych operacji arytmetycznych rośnie znacząco wraz ze wzrostem parametru k, co wynika bezpośrednio z faktu, że liczba elementów w macierzy zwiększa się wykładniczo.

3. Dla małych wartości k (tj. małych macierzy) liczba operacji oraz wartości FLOPS są niewielkie, ponieważ dominują koszty narzutu rekurencyjnego oraz operacji przygotowawczych, a samo mnożenie obejmuje niewielką liczbę elementów.

4. Wraz ze wzrostem rozmiaru macierzy (większe k) obserwuje się dynamiczny wzrost liczby operacji oraz wartości FLOPS, co jest zgodne z teoretyczną złożonością analizowanych algorytmów.

5. Zastosowanie algorytmu Strassena dla mniejszych podproblemów (dla odpowiednio dobranego progu) pozwala ograniczyć liczbę operacji arytmetycznych w porównaniu do klasycznego podejścia, co potwierdza jego przewagę asymptotyczną.

6. Dobór wartości progu przełączania pomiędzy algorytmem Binéta a Strassena ma istotny wpływ na uzyskane wyniki. Odpowiednio dobrany próg pozwala zminimalizować całkowitą liczbę operacji oraz poprawić wydajność obliczeniową algorytmu.

7. Analiza wartości FLOPS wskazuje, że efektywność obliczeniowa algorytmu zależy zarówno od rozmiaru problemu, jak i od zastosowanej strategii obliczeniowej, w tym wyboru metody mnożenia oraz punktu przełączenia między algorytmami.